A árvore sintática

Gramáticas livres de contexto

Encontro 10 | 30 de setembro

Linguagens Formais e Compiladores

Onde vocês estão



Onde vocês estão



O lexer da E2 está pronto: ele já transforma caracteres em tokens.

Onde vocês estão



O lexer da E2 está pronto: ele já transforma caracteres em tokens.

Hoje: o que o parser faz com esses tokens.

1
Um arquivo, e o vocabulário todo

Um arquivo .g4 completo

```
grammar Calc;

programa : instrucao+ EOF ;
instrucao : IDENT '=' expr ';' ;
expr : expr '*' expr
         | expr '+' expr
         | NUMERO
         | IDENT ;

IDENT : [a-zA-Z_] [a-zA-Z_0-9]* ;
NUMERO : [0-9]+ ;
ESPACO : [ \t\r\n]+ -> skip ;
```

Um arquivo .g4 completo

```
grammar Calc;

programa : instrucao+ EOF ;
instrucao : IDENT '=' expr ';' ;
expr : expr '*' expr
         | expr '+' expr
         | NUMERO
         | IDENT ;

IDENT : [a-zA-Z_] [a-zA-Z_0-9]* ;
NUMERO : [0-9]+ ;
ESPACO : [ \t\r\n]+ -> skip ;
```

Tudo o que vimos no semestre está aqui. Vamos nomear as partes.

Alfabeto

Os símbolos que podem aparecer na entrada.

```
[a-zA-Z_]  [0-9]  =  ;  +  *
```

Alfabeto

Os símbolos que podem aparecer na entrada.

```
[a-zA-Z_]  [0-9]  =  ;  +  *
```

No nosso caso, letras, dígitos e alguns símbolos.
Escreve-se Σ nos livros.

Alfabeto

Os símbolos que podem aparecer na entrada.

```
[a-zA-Z_]  [0-9]  =  ;  +  *
```

No nosso caso, letras, dígitos e alguns símbolos.
Escreve-se Σ nos livros.

É o mesmo Σ do `{a, b}` que usamos a semestre inteiro.
Só que agora com caracteres de verdade.

Palavra e linguagem

palavra	uma sequência de símbolos do alfabeto
linguagem	o conjunto de todas as palavras válidas

Palavra e linguagem

palavra	uma sequência de símbolos do alfabeto
linguagem	o conjunto de todas as palavras válidas

Aqui a "palavra" é um **programa inteiro**:

```
x = 3 + 4;
```

Palavra e linguagem

palavra	uma sequência de símbolos do alfabeto
linguagem	o conjunto de todas as palavras válidas

Aqui a "palavra" é um **programa inteiro**:

```
x = 3 + 4;
```

E a linguagem é o conjunto de todos os programas válidos.

Gramática

O conjunto de regras que **gera** a linguagem.

Gramática

O conjunto de regras que **gera** a linguagem.

O arquivo inteiro que vocês viram é uma gramática.

Gramática

O conjunto de regras que **gera** a linguagem.

O arquivo inteiro que vocês viram é uma gramática.

Cada linha dela chama-se **produção** ou simplesmente regra.

Não terminal: um nome para uma categoria

`expr` não é nada em si. É o **nome** de tudo que pode ser uma expressão.

Não terminal: um nome para uma categoria

`expr` não é nada em si. É o **nome** de tudo que pode ser uma expressão.

E a gramática diz o que cabe nessa categoria:

uma multiplicação	`3 * 4`
uma soma	`2 + 7`
algo entre parênteses	`(x + 1)`
um número	`42`
um identificador	`x`

Não terminal: um nome para uma categoria

`expr` não é nada em si. É o **nome** de tudo que pode ser uma expressão.

E a gramática diz o que cabe nessa categoria:

uma multiplicação	`3 * 4`
uma soma	`2 + 7`
algo entre parênteses	`(x + 1)`
um número	`42`
um identificador	`x`

Nenhum programa contém a palavra `expr`.

Terminal: coisa pronta

`'='` e `';'` não são nome de nada. São os símbolos em si.

Terminal: coisa pronta

`'='` e `';'` não são nome de nada. São os símbolos em si.

Eles aparecem **literalmente** no programa, e não há o que substituir.

Terminal: coisa pronta

`'='` e `';'` não são nome de nada. São os símbolos em si.

Eles aparecem **literalmente** no programa, e não há o que substituir.

Terminal é onde a substituição *termina*.
Daí o nome.

Como saber qual é qual

Olhe a coluna da esquerda, antes dos dois-pontos:

```
programa : instrucao+ EOF ;
instrucao : IDENT '=' expr ';' ;
expr : expr '*' expr | expr '+' expr | NUMERO | IDENT ;
```

Como saber qual é qual

Olhe a coluna da esquerda, antes dos dois-pontos:

```
programa : instrucao+ EOF ;
instrucao : IDENT '=' expr ';' ;
expr : expr '*' expr | expr '+' expr | NUMERO | IDENT ;
```

Três nomes aparecem ali: `programa`, `instrucao`, `expr`.

Como saber qual é qual

Olhe a coluna da esquerda, antes dos dois-pontos:

```
programa : instrucao+ EOF ;
instrucao : IDENT '=' expr ';' ;
expr : expr '*' expr | expr '+' expr | NUMERO | IDENT ;
```

Três nomes aparecem ali: `programa`, `instrucao`, `expr`.

Esses três são os não terminais. Todo o resto é terminal.

O teste, em uma frase

O símbolo tem uma regra própria?

Tem: não terminal. Não tem: terminal.

O teste, em uma frase

O símbolo tem uma regra própria?

Tem: não terminal. **Não tem: terminal.**

Ter regra própria significa aparecer à esquerda dos dois-pontos.

O teste, em uma frase

O símbolo tem uma regra própria?

Tem: não terminal. **Não tem: terminal.**

Ter regra própria significa aparecer à esquerda dos dois-pontos.

É mecânico. Não precisa de intuição.

Aplicando na regra de instrução

```
instrucao :  IDENT '=' expr ';' ;
```

símbolo	**tem regra própria?**	
`expr`	sim	não terminal
`IDENT`	não	terminal
`'='`	não	terminal
`';'`	não	terminal

Aplicando na regra de instrução

```
instrucao :  IDENT '=' expr ';' ;
```

símbolo	tem regra própria?	
`expr`	sim	não terminal
`IDENT`	não	terminal
`'='`	não	terminal
`';'`	não	terminal

Um só precisa ser aberto. Os outros três já são o texto final.

Aplicando na regra de instrução

```
instrucao :  IDENT '=' expr ';' ;
```

símbolo	**tem regra própria?**	
`expr`	sim	não terminal
`IDENT`	não	terminal
`'='`	não	terminal
`';'`	não	terminal

Um só precisa ser aberto. Os outros três já são o texto final.

A derivação acaba quando não sobra nenhum não terminal.

Um aviso sobre a caixa das letras

Nas gramáticas dos livros, maiúscula é não terminal e minúscula é terminal.

Um aviso sobre a caixa das letras

Nas gramáticas dos livros, maiúscula é não terminal e minúscula é terminal.

No ANTLR é o contrário.

Um aviso sobre a caixa das letras

Nas gramáticas dos livros, maiúscula é não terminal e minúscula é terminal.

No ANTLR é o contrário.

no arquivo	é	para o parser
`NUMERO`, `IDENT`	token	terminal
`expr`, `instrucao`	regra de parser	não terminal

Por que faz sentido assim

O lexer já resolveu os tokens antes de o parser começar.

Por que faz sentido assim

O lexer já resolveu os tokens antes de o parser começar.

Então, para o parser, um `NUMERO` é coisa pronta.
Não precisa ser trocado por mais nada.

Por que faz sentido assim

O lexer já resolveu os tokens antes de o parser começar.

Então, para o parser, um `NUMERO` é coisa pronta.
Não precisa ser trocado por mais nada.

É exatamente a definição de terminal: fica e não muda.

O glossário, de uma vez

termo	**no arquivo** `.g4`
alfabeto	os caracteres que aparecem nas regras de lexer
palavra	um programa
linguagem	todos os programas válidos
gramática	o arquivo inteiro
produção	cada regra, cada linha com `:`
terminal	token e literal: `IDENT`, `'='`
não terminal	regra de parser: `expr`, `instrucao`
derivação	aplicar regras até sobrar só terminal

O glossário, de uma vez

termo	**no arquivo** `.g4`
alfabeto	os caracteres que aparecem nas regras de lexer
palavra	um programa
linguagem	todos os programas válidos
gramática	o arquivo inteiro
produção	cada regra, cada linha com `:`
terminal	token e literal: `IDENT`, `'='`
não terminal	regra de parser: `expr`, `instrucao`
derivação	aplicar regras até sobrar só terminal

Todos esses termos apareceram no semestre. Agora estão num lugar só.

2
A gramática do parser

A mesma regra, escrita de duas formas

```
bloco : '{' bloco '}' | 'comando' ; // minuscula: parser
BLOCO : '{' BLOCO '}' | 'comando' ; // MAIUSCULA: lexer
```

A mesma regra, escrita de duas formas

```
bloco : '{' bloco '}' | 'comando' ; // minuscula: parser
BLOCO : '{' BLOCO '}' | 'comando' ; // MAIUSCULA: lexer
```

As duas linhas dizem exatamente a mesma coisa.

A mesma regra, escrita de duas formas

```
bloco : '{' bloco '}' | 'comando' ; // minuscula: parser
BLOCO : '{' BLOCO '}' | 'comando' ; // MAIUSCULA: lexer
```

As duas linhas dizem exatamente a mesma coisa.

a de baixo **o ANTLR recusa**

a de cima **funciona**

Por que uma funciona e a outra não

A regra chama a si mesma: `bloco` aparece dentro de `bloco`.

Por que uma funciona e a outra não

A regra chama a si mesma: `bloco` aparece dentro de `bloco`.

Para executar isso é preciso guardar onde parou
antes de entrar na chamada de dentro.

Por que uma funciona e a outra não

A regra chama a si mesma: `bloco` aparece dentro de `bloco`.

Para executar isso é preciso guardar onde parou
antes de entrar na chamada de dentro.

Guardar onde parou é uma pilha.

Por que uma funciona e a outra não

A regra chama a si mesma: `bloco` aparece dentro de `bloco`.

Para executar isso é preciso guardar onde parou
antes de entrar na chamada de dentro.

Guardar onde parou é uma pilha.

Regra de lexer vira autômato finito, e autômato finito não tem pilha.
Por isso o ANTLR recusa.

A diferença, em uma linha

Regra de parser pode chamar a si mesma.

Regra de lexer não pode.

A diferença, em uma linha

Regra de parser pode chamar a si mesma.
Regra de lexer não pode.

É essa única diferença que permite o aninhamento.

A diferença, em uma linha

Regra de parser pode chamar a si mesma.
Regra de lexer não pode.

É essa única diferença que permite o aninhamento.

Gramáticas com essa liberdade chamam-se **livres de contexto**.

Uma regra que se usa de verdade

```
bloco : '{' bloco '}' // um bloco dentro de outro
      | 'comando' // ou so um comando
      ;
```

Uma regra que se usa de verdade

```
bloco : '{' bloco '}' // um bloco dentro de outro
      | 'comando' // ou so um comando
      ;
```

Leia assim: um bloco é ou um comando,
ou outro bloco entre chaves.

Uma regra que se usa de verdade

```
bloco : '{' bloco '}' // um bloco dentro de outro
      | 'comando' // ou so um comando
      ;
```

Leia assim: um bloco é ou um comando,
ou outro bloco entre chaves.

Repare que `bloco` aparece *dentro* da própria regra.

O que essa regra gera

```
comando
```

O que essa regra gera

```
comando

{comando}
```

O que essa regra gera

```
comando

{comando}

{{comando}}
```

O que essa regra gera

```
comando
{comando}
{{comando}}
{{{comando}}}
```

O que essa regra gera

```
comando

{comando}

{{comando}}

{{{comando}}}
```

Cada volta acrescenta uma chave **de cada lado**.
E nunca sobra nem falta.

O que essa regra gera

```
comando
{comando}
{{comando}}
{{{comando}}}
```

Cada volta acrescenta uma chave **de cada lado**.
E nunca sobra nem falta.

É exatamente o aninhamento que vocês escrevem em código.

Por que autômato finito não dá conta

Para fechar direito, é preciso lembrar **quantas** chaves foram abertas.

Por que autômato finito não dá conta

Para fechar direito, é preciso lembrar **quantas** chaves foram abertas.

E não há limite para esse número.

Por que autômato finito não dá conta

Para fechar direito, é preciso lembrar **quantas** chaves foram abertas.

E não há limite para esse número.

Foi isso que vocês descobriram no encontro 7, escrevendo o validador:
o contador precisava crescer sem teto.

Uma gramática de expressões

```
expr : expr '*' expr
     | expr '+' expr
     | '(' expr ')'
     | NUMERO
     ;
```

Uma gramática de expressões

```
expr : expr '*' expr
     | expr '+' expr
     | '(' expr ')'
     | NUMERO
     ;
```

Repare que `expr` aparece **dentro** da própria definição.

Uma gramática de expressões

```
expr : expr '*' expr
     | expr '+' expr
     | '(' expr ')'
     | NUMERO
     ;
```

Repare que `expr` aparece **dentro** da própria definição.

É essa recursão que permite expressões de qualquer tamanho.

3

O que o parser produz

Não é sim ou não

O lexer respondia: este texto é um `IDENT`? Sim ou não.

Não é sim ou não

O lexer respondia: este texto é um `IDENT`? Sim ou não.

O parser responde: *como* estes tokens se encaixam.

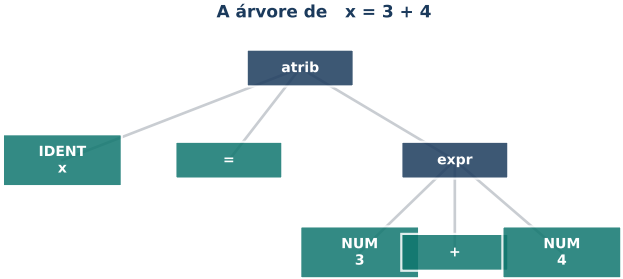
Não é sim ou não

O lexer respondia: este texto é um `IDENT`? Sim ou não.

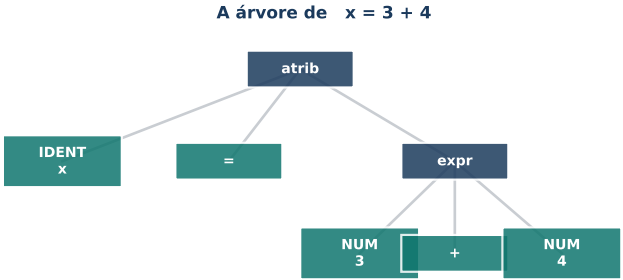
O parser responde: *como* estes tokens se encaixam.

E a resposta é uma **árvore**.

A árvore sintática

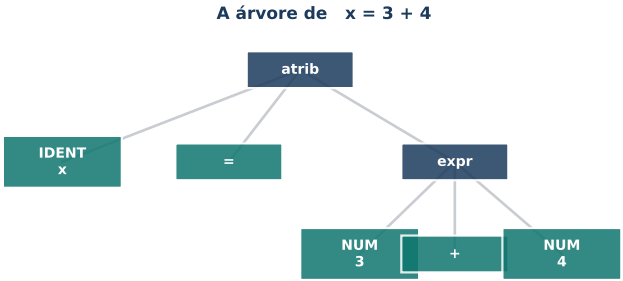


A árvore sintática



As folhas são os tokens. Os nós internos são as regras da gramática.

A árvore sintática

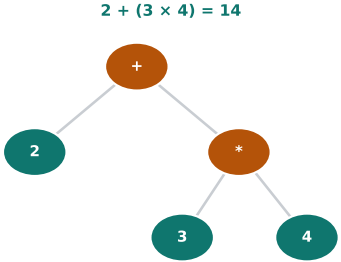


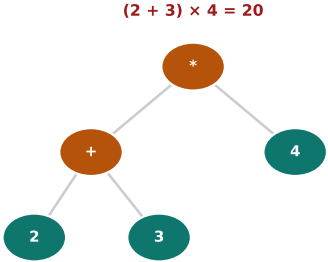
As folhas são os tokens. Os nós internos são as regras da gramática.

É essa árvore que as próximas fases do compilador percorrem.

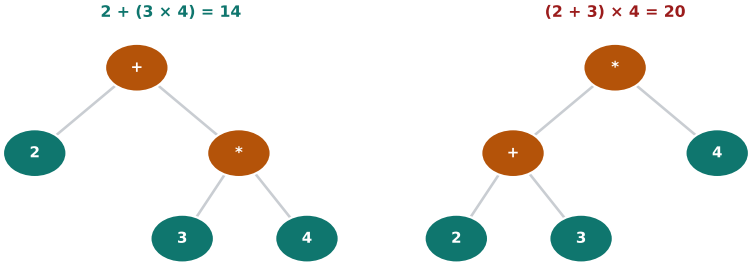
4
O perigo

A mesma expressão, duas árvores



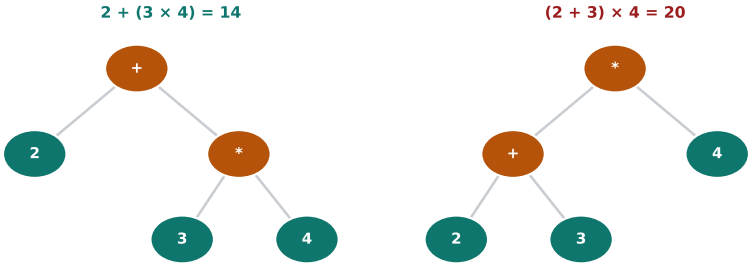


A mesma expressão, duas árvores



A entrada é a mesma: 2 + 3 * 4.

A mesma expressão, duas árvores



A entrada é a mesma: 2 + 3 * 4.

Resultados diferentes: 14 e 20.

Isso se chama ambiguidade

Uma gramática é **ambígua** quando a mesma entrada admite mais de uma árvore.

Isso se chama ambiguidade

Uma gramática é **ambígua** quando a mesma entrada admite mais de uma árvore.

E aí o compilador não sabe qual escolher.

Isso se chama ambiguidade

Uma gramática é **ambígua** quando a mesma entrada admite mais de uma árvore.

E aí o compilador não sabe qual escolher.

Não é um erro que dá mensagem. É um programa que calcula errado.

E não são só duas

operandos na expressão	árvores possíveis
3	2
4	5
5	14
6	**42**

E não são só duas

operandos na expressão	árvores possíveis
3	2
4	5
5	14
6	**42**

Sem uma regra que decida, `a+b*c-d/e+f` tem 42 leituras possíveis.

A solução no ANTLR: a ordem

```
expr : expr '*' expr // declarada primeiro
     | expr '+' expr // liga mais fraco
     | NUMERO ;
```

A solução no ANTLR: a ordem

```
expr : expr '*' expr // declarada primeiro
     | expr '+' expr // liga mais fraco
     | NUMERO ;
```

A alternativa declarada **antes** liga mais forte.

`*` antes de `+`	`2+3*4` vira `2+(3*4)` $= 14$
`+` antes de `*`	`2+3*4` vira `(2+3)*4` $= 20$

A solução no ANTLR: a ordem

```
expr : expr '*' expr // declarada primeiro
     | expr '+' expr // liga mais fraco
     | NUMERO ;
```

A alternativa declarada **antes** liga mais forte.

* antes de +	2+3*4 vira 2+(3*4) = 14
+ antes de *	2+3*4 vira (2+3)*4 = 20

Trocar duas linhas de lugar muda o resultado dos programas.

A ordem correta

Do que liga mais forte para o que liga mais fraco:

1. parênteses
2. sinal de menos unário
3. `*` `/`
4. `+` `-`
5. comparações
6. `e` `ou`

A ordem correta

Do que liga mais forte para o que liga mais fraco:

1. parênteses
2. sinal de menos unário
3. `*` `/`
4. `+` `-`
5. comparações
6. `e` `ou`

É a mesma ordem da matemática, e de quase toda linguagem. Sigam essa a não ser que tenham motivo para mudar.

O que fica

- Regra de parser pode ter o não terminal em qualquer lugar.

O que fica

- Regra de parser pode ter o não terminal em qualquer lugar.
- É isso que permite aninhamento, e define as livres de contexto.

O que fica

- Regra de parser pode ter o não terminal em qualquer lugar.
- É isso que permite aninhamento, e define as livres de contexto.
- O parser não responde sim ou não: ele produz uma árvore.

O que fica

- Regra de parser pode ter o não terminal em qualquer lugar.
- É isso que permite aninhamento, e define as livres de contexto.
- O parser não responde sim ou não: ele produz uma árvore.
- Gramática ambígua admite mais de uma árvore para a mesma entrada.

O que fica

- Regra de parser pode ter o não terminal em qualquer lugar.
- É isso que permite aninhamento, e define as livres de contexto.
- O parser não responde sim ou não: ele produz uma árvore.
- Gramática ambígua admite mais de uma árvore para a mesma entrada.
- No ANTLR, a alternativa declarada antes liga mais forte.

O que fica

- Regra de parser pode ter o não terminal em qualquer lugar.
- É isso que permite aninhamento, e define as livres de contexto.
- O parser não responde sim ou não: ele produz uma árvore.
- Gramática ambígua admite mais de uma árvore para a mesma entrada.
- No ANTLR, a alternativa declarada antes liga mais forte.
- Trocar duas linhas de lugar muda o resultado dos programas.

O que fica

- Regra de parser pode ter o não terminal em qualquer lugar.
- É isso que permite aninhamento, e define as livres de contexto.
- O parser não responde sim ou não: ele produz uma árvore.
- Gramática ambígua admite mais de uma árvore para a mesma entrada.
- No ANTLR, a alternativa declarada antes liga mais forte.
- Trocar duas linhas de lugar muda o resultado dos programas.

Agora: escrevam as regras de parser da linguagem de vocês.